

SOLID Principles

Writing maintainable, scalable TypeScript

What is SOLID?

Five principles for **clean, maintainable software design** by Robert C. Martin (Uncle Bob).

Letter	Principle	Core Idea
S	Single Responsibility	One reason to change
O	Open/Closed	Open to extend, closed to modify
L	Liskov Substitution	Subtypes must be substitutable
I	Interface Segregation	No forced unused methods
D	Dependency Inversion	Depend on abstractions

S – Single Responsibility Principle

A class should have only one reason to change.

SRP – The Problem

```
class UserService {  
  createUser(data: any) {  
    // Save to DB  
  }  
  
  sendWelcomeEmail(email: string) {  
    // Send email  
  }  
  
  logToFile(message: string) {  
    // Write to local file  
  }  
}
```

Three unrelated responsibilities in one class – persistence, email, and logging.

A change to any single concern risks breaking the others.

Also makes it harder to test user creation, sending email and logging as they are tightly coupled

SRP – The Solution

```
class UserRepository {
    create(data: any) { console.log("User saved:", data); }
}

class EmailService {
    sendWelcome(email: string) { console.log("Welcome email sent:", email); }
}

class Logger {
    log(message: string) { console.log("[LOG]", message); }
}

class UserService {
    constructor(
        private repo: UserRepository,
        private email: EmailService,
        private logger: Logger
    ) {}

    registerUser(data: any) {
        this.repo.create(data);
        this.email.sendWelcome(data.email);
        this.logger.log("User registered: " + data.email);
    }
}
```

O – Open/Closed Principle

Open for extension, closed for modification.

OCP – The Problem

```
class Payment {  
  pay(type: string) {  
    if (type == "paypal") {  
      // ...  
    }  
    if (type == "stripe") {  
      // ...  
    }  
    // Adding a new method = modifying this class ✖  
  }  
}
```

Every new payment method requires changing existing code – risking regressions.

Also makes it harder devs to work in parallel on different payment methods

OCP – The Solution

```
interface PaymentStrategy {  
    pay(amount: number): void;  
}  
  
class PayPalPayment implements PaymentStrategy {  
    pay(amount: number) { console.log("Paid with PayPal:", amount); }  
}  
  
class StripePayment implements PaymentStrategy {  
    pay(amount: number) { console.log("Paid with Stripe:", amount); }  
}  
  
class Checkout {  
    constructor(private payment: PaymentStrategy) {}  
  
    process(amount: number) {  
        this.payment.pay(amount);  
    }  
}
```

New payment methods = new classes. Existing code is **never touched**.

Can use a different instance of Checkout for different payment methods

L – Liskov Substitution Principle

Subtypes must be replaceable for their base types without breaking behaviour.

LSP – The Problem

```
class Bird {  
    fly() {}  
}  
  
class Penguin extends Bird {  
    fly() {  
        throw new Error("Penguins cannot fly!"); // ✖ Breaks caller expectations  
    }  
}
```

Code that expects a `Bird` will **crash** when given a `Penguin`.

Inheritance modelled incorrectly – forced behaviour that doesn't exist.

If the class can't be substituted for the same as its parent class, it won't work

LSP – The Solution

```
interface Flyable {  
    fly(): void;  
}  
  
class Sparrow implements Flyable {  
    fly() { console.log("Flying ... "); }  
}  
  
class Penguin {  
    swim() { console.log("Swimming ... "); }  
}
```

Model **capabilities**, not hierarchy. Each class only promises what it can deliver.

Usually solved by using an interface over a class for stating the type for an object instance, but can still violate this principle

I – Interface Segregation Principle

Clients should not be forced to depend on interfaces they don't use.

ISP – The Problem

```
interface Worker {  
    work(): void;  
    eat(): void;  
}  
  
class Robot implements Worker {  
    work() {}  
    eat() {  
        throw new Error("Robots do not eat"); // ❌ Irrelevant contract  
    }  
}
```

Robot is forced to implement a method that makes no sense for it – a **fat interface**.

ISP – The Solution

```
interface Workable {  
    work(): void;  
}  
  
interface Eatable {  
    eat(): void;  
}  
  
class Human implements Workable, Eatable {  
    work() {}  
    eat() {}  
}  
  
class Robot implements Workable {  
    work() {}  
}
```

Small, focused interfaces. Each class implements **only what it needs**.

D – Dependency Inversion Principle

High-level modules should not depend on low-level modules. Both should depend on abstractions.

DIP – The Problem

```
class MySQLDatabase {  
    save(data: any) { console.log("Saving to MySQL", data); }  
}  
  
class UserService {  
    db = new MySQLDatabase(); // ❌ Hard dependency on a concrete class  
  
    saveUser(data: any) {  
        this.db.save(data);  
    }  
}
```

Switching the database requires **modifying** `UserService` . Impossible to test in isolation.

What happens if the client wants use to move to s different SQL db technology? it happens :(

DIP – The Solution

```
interface Database {  
    save(data: any): void;  
}  
  
class MySQLDatabase implements Database {  
    save(data: any) { console.log("Saved in MySQL:", data); }  
}  
  
class PostgresDatabase implements Database {  
    save(data: any) { console.log("Saved in PostgreSQL:", data); }  
}  
  
class UserService {  
    constructor(private db: Database) {}  
  
    saveUser(data: any) {  
        this.db.save(data);  
    }  
}
```

`UserService` depends on an **interface**. Swap any database – no changes needed.

Putting It All Together

```
interface MessageSender {
    send(message: string): void;
}

class EmailSender implements MessageSender {    // OCP: extendable via new senders
    send(message: string) { console.log("Email:", message); }
}

class SmsSender implements MessageSender {
    send(message: string) { console.log("SMS:", message); }
}

class NotificationService {                    // DIP: depends on abstraction
    constructor(private sender: MessageSender) {}

    notify(user: string, message: string) {
        this.sender.send(`${user}: ${message}`);
    }
}

// LSP: any MessageSender is safely substitutable
const service = new NotificationService(new EmailSender());
service.notify("Alice", "SOLID principles in action!");
```

SOLID – Summary

Principle	Key Benefit
SRP	Focused classes, easy to test
OCP	Add features without breaking existing code
LSP	Safe, predictable inheritance
ISP	Lean interfaces, no unnecessary coupling
DIP	Swap implementations freely

SOLID principles work together – each reinforces the others.

Key Takeaways

- **SRP** – one class, one job
- **OCP** – extend with new code, don't modify old code
- **LSP** – subtypes must honour the parent contract
- **ISP** – keep interfaces small and focused
- **DIP** – program to interfaces, not implementations

TypeScript's **type system** makes applying SOLID natural and enforceable.

Time to Practice!

Work through the exercises in [part-4/](#) to apply each principle hands-on.